Simulation for Sparsity and CS

Ranim Tom

November 5, 2026

Contents

Todo list

OMP Explanation 9
KPI 9

Chapter 1

Greedy Algorithm

1.1 Perminilinary

Before starting the examples, let's make a small recall about the problem formulation and notation used in CS problem.
The modeling of the CS problem is embedded via **??**

$$\begin{aligned} \vec{y}_{m\times 1} &= \boldsymbol{A}_{m\times n} \times \vec{x}_{n\times 1} \\ &= \boldsymbol{\Phi}_{m\times n} \times \boldsymbol{\Psi}_{n\times n} \times \vec{x}_{n\times 1} \end{aligned} \tag{1.1}$$

where

- $\boldsymbol{A}$: is the sensing matrix
- $\boldsymbol{\Phi}$: measurement matrix
- $\boldsymbol{\Psi}$: basis matrix

The $\vec{x}_{n\times 1}$ is a sparse vector in some domain (frequency,time,$\cdots$). If it is not sparse we should ***apply some sparse transformation***, like DFT to some time domain vector (as we will do in the example).

1.2 Example 1: 3 Spike Component

Source: See section 3.2, example 1 in [**?**], and [**?**] for extra examples and details for implementation (how to implement random matrices and do random selection).

1.2.1 Preprocessing

- A continuous time signal is being taken shown in **??**

$$x(t) = 0.3 + 0.4\exp(-j12\,\pi\,t) + 0.5\,\exp(-j50\,\pi\,t) \tag{1.2}$$

- The signal is being sampled at $F_s = 64$ Hz for 1 s duration, resulting in a total number of samples of 64

- We choose number of measurement in ***time domain*** is $m = 16$, ***taking randomly***
- Since is our initial sampled time domain signal is not sparse, we need to sparsify it, by applying a DFT transform $\rightarrow$ we obtain a sparse frequency signal
- The random measurement will be taken in the time domain, so **??** is applied as follows (for illustration I take $m = 2$ and $n = 5$):

$$\begin{aligned}\vec{y}_{m\times 1} &= \boldsymbol{A}_{m\times n} \times \vec{x}_{n\times 1} \\ &= \boldsymbol{\Phi}_{m\times n} \times \boldsymbol{\Psi}_{n\times n} \times \vec{x}_{n\times 1}\end{aligned}$$

$$= \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 1 & 0 \end{bmatrix}_{m\times n} \times \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 1 & 0 \\ 0 & 0 & 0 & \cdots & 0 \end{bmatrix}_{n\times n} \times \boldsymbol{F}^{-1}_{n\times n} \times \vec{x}_{n\times 1} \tag{1.3}$$

Here, the measurement matrix $\boldsymbol{A}$ is ***random inverse partial DFT matrix***.

Always remember that ***measurement matrix*** $\boldsymbol{A}$ are ***wide matrices***.

1.2.2 Displaying Some Results

In **??**, we see a run trial using OMP algorithm (explained next section).

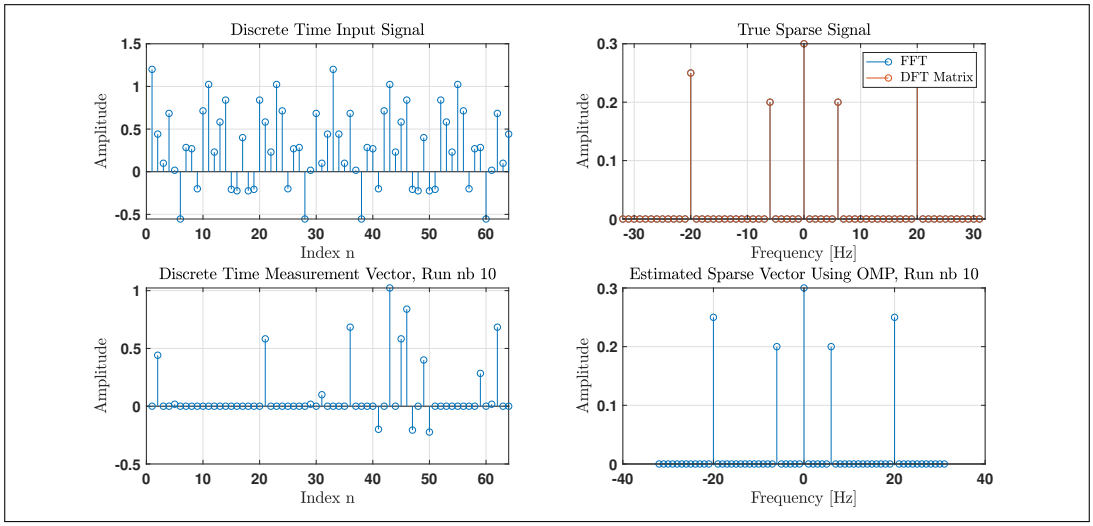


Figure 1.1: Simulation Output

- 1st column: the sampled signal and the random measurement taken
- 2nd column: the true sparse vector (implemented using and FFT and DFT) and the estimated one.

1.2.3 OMP Explanation

IP Explana-
n
:

OMP Explanation:

To explain it later with picture of matrices, update the measurement of matrix according to update support,· · · .

Use idea of slides of Michael Elad.

- OMP Algorithm:

  1. Search for the best atom in $\boldsymbol{A}$ that explain $\vec{y} = \boldsymbol{A}\vec{x}$
     - $\rightarrow \vec{\rho} = \boldsymbol{A}^T \vec{r}$, where $\vec{\rho}$ is the correlation vector and $\vec{r}$ is the residual vector
     - Taking the index of the maximum value in $\vec{\rho}$
  2. Update the support of indices: $S_k : S_k = S_{k-1} \cup \{i_0\}$
  3. We solve for the values of $\vec{x}_k$ subject to the updated indices, using a least square problem

$$\vec{x}_k = \min_{\vec{x}_k} \|A\,\vec{x} - \vec{y}\|_2 \quad \text{s.t} \quad \text{Supp}\{\vec{x}\} = S_k \tag{1.4}$$

  4. Update the $\vec{r} \rightarrow \vec{r} = \vec{y} - \boldsymbol{A}\,\vec{x}_k$

1.2.4 KPI

As KPI, we can use the following:
KPI

KPI:
To redo them later with analysis. Maybe after finishing the algorithm here the analysis part of Elad.

1. Relative error:

$$\text{Error}_{L_2} = \frac{\|\hat{\vec{x}} - \vec{x}\|_2^2}{\|\vec{x}\|_2^2} \tag{1.5}$$

2. We can also asses support recovery based on the normalized intersection of the true and the estimated support by applying the following equation:

$$\text{Error}_S = 1 - \frac{|\hat{S}| \cap |S|}{\max\left\{|\hat{S}|\,,\,|S|\right\}} \tag{1.6}$$

   If the 2 support don't overlap, their intersection is 0 and hence Error_S is equal to 1

Bibliography

[1] L. Stankovic, E. Sejdic, S. Stankovic, M. Dakovic, and I. Orovic, "A tutorial on sparse signal reconstruction and its applications in signal processing," *Circuits, Systems, and Signal Processing*, vol. 38, 03 2019.

[2] G. Peyré, "The numerical tours of signal processing," *Computing in Science Engineering*, vol. 13, no. 4, pp. 94–97, 2011.